18.4 Telegram 草稿流

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~520k tokens，输出 ~45k tokens（本章合计）

前几节讲的是通用的流式传输架构和块分割算法。这些机制最终要落地到具体的通道上——用户在 Telegram 上看到的“实时打字”效果，正是由一个名为 **Telegram 草稿流（Draft Stream）** 的子系统实现的。

Telegram Bot API 原生并不支持"边生成边更新消息"的功能。OpenClaw 利用了 `sendMessageDraft` 这一 API 端点，通过不断发送"草稿"来模拟流式打字效果。本节将详细分析这一机制的实现，以及 partial 模式与 block 模式两种不同的更新策略。

18.4.1 `sendMessageDraft` 的实现

草稿流的核心抽象

Telegram 草稿流的核心是 `createTelegramDraftStream` 工厂函数，位于 `src/telegram/draft-stream.ts` 。它接受一组参数并返回一个 `TelegramDraftStream` 对象，该对象只暴露三个方法：

```
// src/telegram/draft-stream.ts
export type TelegramDraftStream = {
  update: (text: string) => void;   // 更新草稿内容
  flush: () => Promise<void>;       // 强制刷新缓冲区
  stop: () => void;                 // 停止草稿流
};
```

这个接口设计得很克克——上层调用者只需要关心三个操作，节流、冲突检测、长度限制等内部细节全部封装在内部。

关键常量与参数

工厂函数在创建时会固化以下参数：

参数	默认值	说明
`maxChars`	4096	Telegram 消息长度上限
`throttleMs`	300ms	发送节流间隔（最小 50ms）
`draftId`	消息 ID	关联的原始消息标识，非零正整数
`chatId`	—	目标对话 ID

```
// src/telegram/draft-stream.ts
const TELEGRAM_DRAFT_MAX_CHARS = 4096;
const DEFAULT_THROTTLE_MS = 300;

const maxChars = Math.min(params.maxChars ?? TELEGRAM_DRAFT_MAX_CHARS, TELEGRAM_DRAFT_M
const throttleMs = Math.max(50, params.throttleMs ?? DEFAULT_THROTTLE_MS);
const rawDraftId = Number.isFinite(params.draftId) ? Math.trunc(params.draftId) : 1;
const draftId = rawDraftId === 0 ? 1 : Math.abs(rawDraftId);
```

注意 `draftId` 的防御性处理：截断为整数、确保非零、取绝对值。这是因为 Telegram API 对草稿 ID 有严格的格式要求。

内部状态机

草稿流内部维护 5 个状态变量，构成一个隐式的**有限状态机**：

```
┌──────────────────────────────────────────┐
│            Draft Stream State            │
├──────────────────────────────────────────┤
│  lastSentText  : string  = ""            │  ← 去重用：上次实际发送的文本
│  lastSentAt    : number  = 0             │  ← 节流用：上次发送的时间戳
│  pendingText   : string  = ""            │  ← 缓冲用：等待发送的最新文本
│  inFlight      : boolean = false         │  ← 互斥用：是否有请求正在进行
│  stopped       : boolean = false         │  ← 终止用：是否已停止
└──────────────────────────────────────────┘
```

> **衍生解释：有限状态机（FSM）**

> 有限状态机是一种数学模型，用有限数量的状态和状态之间的转换来描述系统行为。在这里，虽然没有显式的 `enum State { ... }`，但这几个布尔量和字符串变量的组合隐式地定义了草稿流可能处于的所有状态。例如 `stopped=true` 时，任何 `update()` 调用都是空操作；`inFlight=true` 时，新的 `flush()` 只会调度下一次发送而不会立即执行。

发送逻辑：`sendDraft`

内部的 `sendDraft` 函数负责实际的 API 调用：

```
// src/telegram/draft-stream.ts（简化）
const sendDraft = async (text: string) => {
  if (stopped) return;
  const trimmed = text.trimEnd();
  if (!trimmed) return;

  // 超长则永久停止——不发截断版本也不发失败版本
  if (trimmed.length > maxChars) {
    stopped = true;
    params.warn?.(`draft length ${trimmed.length} > ${maxChars}`);
    return;
  }

  // 去重：与上次发送相同则跳过
  if (trimmed === lastSentText) return;

  lastSentText = trimmed;
  lastSentAt = Date.now();
  try {
    await params.api.sendMessageDraft(chatId, draftId, trimmed, threadParams);
  } catch (err) {
    stopped = true;  // API 失败时永久停止，不重试
    params.warn?.(`draft stream failed: ${err instanceof Error ? err.message : String(e
  }
};
```

这里有两个设计决策值得一提：

1. **超长即停**：当文本超过 4096 字符时，不会尝试截断后发送，而是直接停止整个草稿流。这是因为截断可能会破坏 Markdown 结构（如未闭合的代码块），产生比"不更新"更差的用户体验。
2. **失败即停**：API 调用失败时也永久停止，而不是重试。这避免了在网络不稳定时持续产生失败请求。最终回复仍会通过正常的消息发送通道送达，草稿流只是锦上添花的预览功能。

节流调度： update 与 schedule

update 方法是外部最常调用的入口：

```
// src/telegram/draft-stream.ts（简化）
const update = (text: string) => {
  if (stopped) return;
  pendingText = text;    // 始终覆盖，只保留最新

  if (inFlight) {        // 有请求在飞，排队等候
    schedule();
    return;
  }
  if (!timer && Date.now() - lastSentAt >= throttleMs) {
    void flush();        // 节流窗口已过，立即发送
    return;
  }
  schedule();            // 否则安排延迟发送
};

const schedule = () => {
  if (timer) return;     // 已有定时器则不重复安排
  const delay = Math.max(0, throttleMs - (Date.now() - lastSentAt));
  timer = setTimeout(() => { void flush(); }, delay);
};
```

这里的关键设计是 **"覆盖式缓冲"**： pendingText 始终被最新的文本覆盖，而不是追加。这意味着如果在 300ms 的节流窗口内收到了 10 次 update ，只有最后一次的文本会被发送。这正是草稿流的正确语义——用户只需要看到最新的打字进度。

调度流程可以用下图概括：

```
update("Hello")
    │
    ▼
pendingText = "Hello"
    │
    ├─ inFlight? ──────yes──→ schedule() → 等待当前请求完成
    │                                        │
    │                                        ▼
    │                                   flush() → sendDraft()
    │
    ├─ 节流窗口已过? ──yes──→ flush() → sendDraft("Hello")
    │
    └─ 否则 ──────────────→ schedule()
                                 │
                                 ▼
                            setTimeout(flush, delay)
                                 │
                                 ▼
                            flush() → sendDraft("Hello")
```

刷新与并发保护： `flush`

`flush` 方法是节流调度的核心执行器：

```
// src/telegram/draft-stream.ts（简化）
const flush = async () => {
  if (timer) { clearTimeout(timer); timer = undefined; }

  if (inFlight) {        // 防止并发发送
    schedule();          // 推迟到下一轮
    return;
  }

  const text = pendingText;
  const trimmed = text.trim();
  if (!trimmed) {
    pendingText = "";
    if (pendingText) schedule();  // 竞态检查
    return;
  }

  pendingText = "";
  inFlight = true;
  try {
    await sendDraft(text);
  } finally {
    inFlight = false;
  }

  if (pendingText) schedule();   // 发送期间有新数据到达
};
```

`inFlight` 标志充当了一个**单元素互斥锁**：在任意时刻最多只有一个 `sendDraft` 在执行。如果 `flush` 被调用时发现有请求在飞，它不会阻塞等待，而是安排下一轮。这确保了：

- 不会有并行的 API 调用竞争
- 不会因为 `await` 阻塞导致调用方挂起
- 在请求完成后自动处理积压的更新

18.4.2 partial 模式 vs block 模式

草稿流有两种更新策略：**partial 模式**和 **block 模式**。它们对应不同的文本来源和更新方式。

模式选择

在 `bot-message-dispatch.ts` 中，模式由配置项 `streamMode` 决定：

```
// src/telegram/bot-message-dispatch.ts（简化）
const draftStream = canStreamDraft
  ? createTelegramDraftStream({ api: bot.api, chatId, draftId: msg.message_id, ... })
  : undefined;

// block 模式需要额外创建一个 BlockChunker
const draftChunking = draftStream && streamMode === "block"
  ? resolveTelegramDraftStreamingChunking(cfg, route.accountId)
  : undefined;
const draftChunker = draftChunking ? new EmbeddedBlockChunker(draftChunking) : undefine
```

模式的区别可以用下表总结：

特性	partial 模式	block 模式
文本来源	`onPartialReply` 回调（流式增量文本）	`onPartialReply` 回调 + BlockChunker 分块
更新方式	直接替换整个草稿	增量追加，经过 BlockChunker 分段
更新频率	每个 token 产出后	按段落/代码块边界
用户体验	逐字出现，类似"打字机"	按段落出现，更整洁
适用场景	短对话，即时反馈	长文本生成，结构化输出

partial 模式的数据流

在 partial 模式下，数据流非常直接：

```
// src/telegram/bot-message-dispatch.ts（简化）
const updateDraftFromPartial = (text?: string) => {
  if (!draftStream || !text) return;
  if (text === lastPartialText) return;

  if (streamMode === "partial") {
    lastPartialText = text;
    draftStream.update(text);      // 直接用完整文本更新草稿
    return;
  }
  // ...block 模式逻辑见下...
};
```

partial 模式下，每次收到 `onPartialReply` 回调时：

1. 完整的已生成文本直接传入 `draftStream.update()`
2. 草稿流内部节流后发送给 Telegram API
3. 用户在对话中看到文本逐字增长

数据流图：

```
模型输出 delta
    │
    ▼
handleMessageUpdate
    │ 拼接到 deltaBuffer
    ▼
stripBlockTags + parseReplyDirectives
    │ 清理后的完整文本
    ▼
onPartialReply({ text })
    │
    ▼
updateDraftFromPartial(text)
    │ streamMode === "partial"
    ▼
draftStream.update(text)
    │ 节流 (300ms)
    ▼
sendMessageDraft(chatId, draftId, text)
    │
    ▼
用户在 Telegram 中看到实时预览
```

block 模式的数据流

block 模式更复杂——它在草稿流之前插入了一个独立的 `EmbeddedBlockChunker` （注意这与 subscribe 层的 blockChunker 是**不同的实例**）：

```typescript
// src/telegram/bot-message-dispatch.ts（简化）
const updateDraftFromPartial = (text?: string) => {
  if (!draftStream || !text) return;
  if (text === lastPartialText) return;

  // block 模式
  let delta = text;
  if (text.startsWith(lastPartialText)) {
    delta = text.slice(lastPartialText.length);  // 提取增量
  } else {
    // 非单调流（流缓冲区重置），从头开始
    draftChunker?.reset();
    draftText = "";
  }
  lastPartialText = text;
  if (!delta) return;

  if (!draftChunker) {
    // 没有 chunker 时退化为 partial 模式
    draftText = text;
    draftStream.update(draftText);
    return;
  }

  // 增量喂入 chunker，按段落边界输出
  draftChunker.append(delta);
  draftChunker.drain({
    force: false,
    emit: (chunk) => {
      draftText += chunk;
      draftStream.update(draftText);
    },
  });
};
```

block 模式的核心差异在于：

1. **增量提取**：从完整文本中计算出增量 `delta` （如果文本是单调增长的话，只需取差集）
2. **BlockChunker 缓冲**：增量先进入 chunker，直到遇到段落边界才释放
3. **草稿追加**：释放的分块追加到 `draftText` ，然后更新草稿

这意味着用户在 block 模式下不会看到逐字出现的效果，而是看到一段段文字"弹出"——更整洁，也更适合长文本。

非单调流的处理

注意代码中对"非单调流"的特殊处理：

```
if (text.startsWith(lastPartialText)) {
  delta = text.slice(lastPartialText.length);
} else {
  // 非单调：重置状态
  draftChunker?.reset();
  draftText = "";
}
```

> **衍生解释：单调流（Monotonic Stream）**
>
> 在流式系统中，如果每一帧数据都是前一帧的"前缀扩展"（即新文本是旧文本加上一段增量），我们称这个流是单调的。OpenClaw 的模型输出通常是单调的——每个 `text_delta` 追加到已有文本之后。但某些情况下单调性会被打破：
>
> - 上下文压缩后重试（compaction retry），模型重新生成
> - 某些模型提供者的异常行为
>
> 当检测到非单调时，必须重置所有增量状态，从头开始构建草稿。

草稿流的生命周期

草稿流在整个回复处理过程中的生命周期如下：

```
用户发送消息
    │
    ▼
canStreamDraft? ──no──→ 不使用草稿流
    │ yes
    ▼
createTelegramDraftStream()
    │
    ▼
┌──────────────────────────┐
│  流式生成阶段              │
│  onPartialReply → update() │  ← 反复调用
│  onPartialReply → update() │
│  ...                       │
└──────────────────────────┘
    │
    ▼
最终回复准备发送
    │
    ▼
flushDraft()                    ← 刷新剩余缓冲
    │
    ▼
draftStream.stop()              ← 停止草稿流
    │
    ▼
deliverReplies()                ← 发送正式消息
```

需要注意的是，**草稿流只在私聊（private chat）中启用**：

```
const canStreamDraft =
  streamMode !== "off" &&
  isPrivateChat &&                    // 仅私聊
  typeof draftThreadId === "number" &&
  (await resolveBotTopicsEnabled(primaryCtx));
```

这是因为群聊中的草稿更新可能会产生过多的通知噪音，影响其他用户。此外，草稿启用时，subscribe 层的 block streaming 会被**禁用**（`disableBlockStreaming = true`），避免两个流式机制冲突：

```
const disableBlockStreaming =
  Boolean(draftStream) ||    // 有草稿流则禁用 block streaming
  (typeof telegramCfg.blockStreaming === "boolean" ? !telegramCfg.blockStreaming : unde
```

本节小结

1. **Telegram 草稿流**通过 `sendMessageDraft` API 实现了"实时打字"效果，是一个独立于通用流式传输架构的通道特化层。
2. **核心设计**采用覆盖式缓冲 + 时间节流 + 单元素互斥锁，确保 API 调用既不过于频繁也不会并发冲突。
3. **超长即停**、**失败即停**的策略优先保证系统稳定性——草稿只是预览，最终回复通过正常通道送达。
4. **partial 模式**直接将完整文本传入草稿流，适合短对话的"打字机"效果。
5. **block 模式**在草稿流前插入独立的 BlockChunker，按段落边界更新草稿，适合长文本的结构化输出。
6. 草稿流仅在**私聊**中启用，并且会自动**禁用** subscribe 层的 block streaming，避免两个流式机制冲突。